

Encoding Categorical Data for Machine Learning

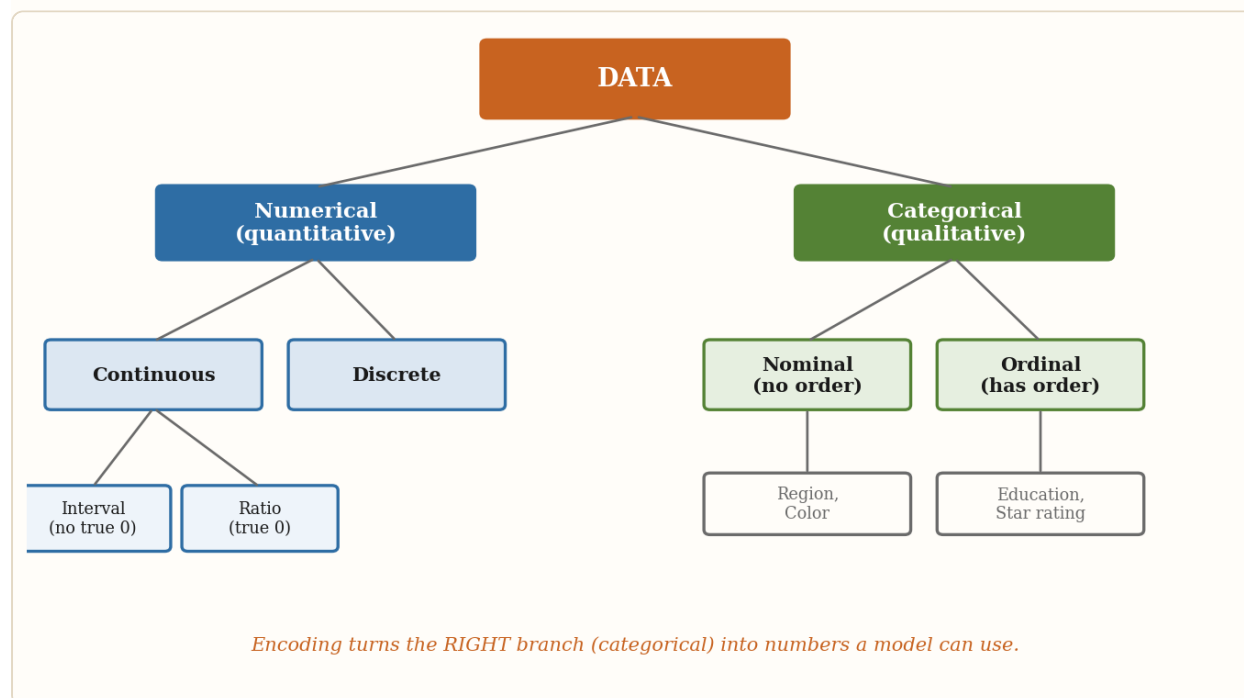
Turning text categories and raw numbers into model-ready values — a beginner's walkthrough with worked code

Nominal · Ordinal · Label · Binary · Frequency · Target encoding · Discretization · Random Forest

1 Why encode categorical data?

Most machine-learning algorithms do arithmetic, so they need their inputs in a numeric form. A column like *Region* that holds the words `APAC`, `Americas`, and `Europe` means nothing to the math until we convert it. **Encoding** is that conversion: it turns qualitative categories into numeric values a model can learn from.

The right encoding depends on the *type* of data. The map below shows where categorical data sits, and the two flavours — **nominal** (no order) and **ordinal** (a real order) — that decide which technique to reach for.



KEY IDEA

Nominal categories have no ranking (Region, Color) — one-hot is the usual choice. **Ordinal** categories have a real order (Education, star ratings) — ordinal encoding fits. Choosing the wrong one can teach the model an order that isn't there.

Encoding matters for three practical reasons the course highlights: models only read numbers; clean encoding makes patterns *easier* to detect; and good encoding reduces **representation bias**, so no category is accidentally treated as bigger or more important than another. (It does not, by itself, remove bias in the broader sense — biased data still yields biased models.)

2 One-hot encoding (for nominal data)

One-hot encoding creates one new **binary column per category**. For each row, the column that matches its category gets a `1` and the rest get `0`. No category outranks another, so the model is never handed a fake order. (It does add redundant columns — handled just below with `drop='first'`.)

BEFORE		AFTER (one-hot)		
Color		Red	Green	Blue
Red	→	1	0	0
Green		0	1	0
Blue		0	0	1
Red		1	0	0

```

ohe = OneHotEncoder(drop="first", sparse_output=False,
                    handle_unknown="ignore")
region_train = ohe.fit_transform(X_train[["Region"]]) # learn + apply on TRAIN
region_test  = ohe.transform(X_test[["Region"]])      # apply only on TEST

```

The trade-off is **dimensionality**: one column becomes many. With `drop="first"` we drop one redundant column (the dropped category becomes the *reference*) to avoid **multicollinearity** — perfect redundancy between columns that upsets models like linear and logistic regression.

WATCH OUT

One-hot on a **high-cardinality** feature (many distinct values) can add hundreds of columns and invite overfitting. For those, consider binary, frequency, or target encoding.

3 Ordinal encoding (for ordered data)

When categories have a genuine order, **ordinal encoding** maps them to ordered integers so the ranking is preserved:

`High School < Bachelor < Master < PhD` becomes `0 < 1 < 2 < 3`. Unlike one-hot, it adds **no new columns**.

```

edu_order = ["High School", "Bachelor", "Master", "PhD"]
oe = OrdinalEncoder(categories=edu_order) # pass the order explicitly
edu_train = oe.fit_transform(X_train[["Education"]]) # High School->0 ... PhD->3

```

GOOD PRACTICE

Always pass the order explicitly with `categories=[...]`. If you don't, the encoder may fall back on alphabetical order and quietly scramble your ranking.

4 Label encoding (for the target)

Label encoding also assigns a unique integer per category, but *without* caring about order. Its best use is the **target** variable — here `"no" → 0` and `"yes" → 1`.

```
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)      # "no"→0, "yes"→1
y_test_enc = le.transform(y_test)
```

DON'T

Avoid label encoding on **nominal input features**: turning Region into 0/1/2 implies Europe > Americas > APAC, which distance-based models (k-NN, k-means) will believe. Tree-based models are *less* sensitive because they split rather than measure distance, but the order is still arbitrary and can occasionally affect where splits land — so it is a convenience, not a default.

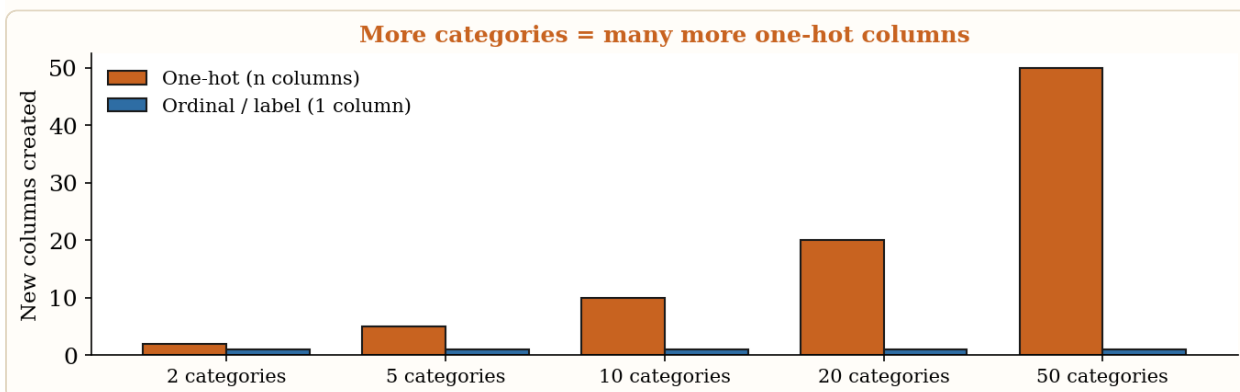
5 The other three: binary, frequency, target

The course covers six encoders in total. Three more fill the gaps left by the first three — mostly for **high-cardinality** features:

Binary encoding writes each category as a binary number across a few columns — roughly $\log_2(n)$ columns instead of n . It curbs the column explosion of one-hot, at the cost of hiding some category relationships.

Frequency (count) encoding replaces a category with how often it occurs. Useful when rarity itself is informative (fraud, churn), but two categories with equal counts *collide* into the same value.

Target encoding replaces a category with a statistic of the target (often its mean). Powerful but leakage-prone — it needs cross-validation and smoothing to be safe.

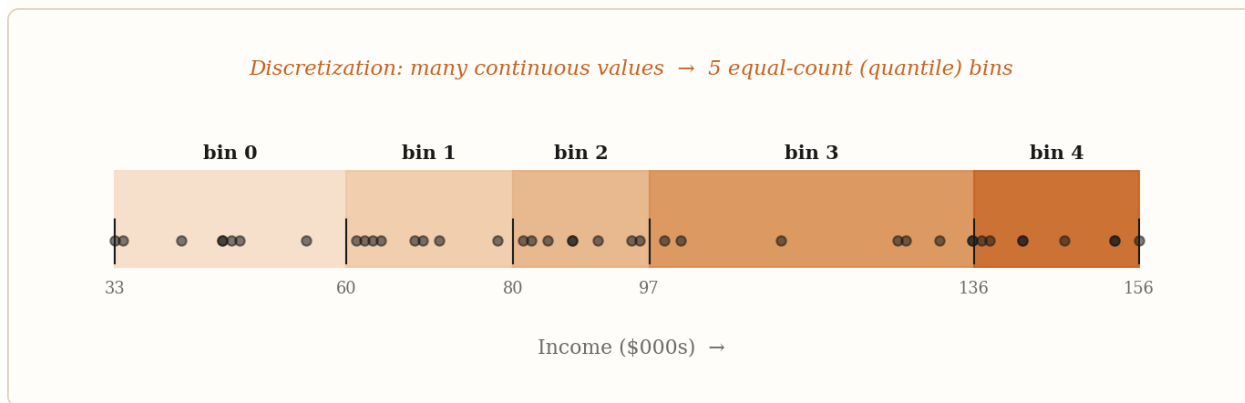


Use this as your cheat-sheet for choosing:

Technique	Data type	Cardinality	Watch out for
One-hot	Nominal	Low–moderate	Explodes columns on high cardinality
Ordinal	Ordinal	Any	Wrong if the order is fake
Label	Target mainly	Any	Arbitrary order — bad for distance models
Binary	Nominal	High	Some category relationships hidden
Frequency / count	Nominal or ordinal	High	Collisions when counts tie
Target	Nominal or ordinal	Low–moderate	Leakage / overfitting without care

6 Discretizing continuous features

Discretization (binning) converts a continuous column such as *Income* into a handful of discrete buckets. The **quantile** strategy makes **equal-count** bins — each holds roughly the same number of rows.



```
disc = KBinsDiscretizer(
    n_bins=5, encode="ordinal", strategy="quantile",
    quantile_method="averaged_inverted_cdf",          # explicit -> no warning
)
num_train = disc.fit_transform(X_train[["Income", "Age"]])  # values -> bin 0..4
```

A HONEST CAVEAT

Binning can smooth noise, but it also **throws away information** (ages 31 and 39 may land in the same bin). Tree models already split continuous values well, so treat binning as something to *validate*, not assume helps.

7 Putting it all together

Here is the full journey the demo runs: encode the categoricals, bin the continuous columns, label-encode the target, then train a Random Forest and score it on unseen rows.



```
rf = RandomForestClassifier(n_estimators=200, random_state=42)
rf.fit(X_train_t, y_train_enc)
predictions = rf.predict(X_test_t)
print(accuracy_score(y_test_enc, predictions))           # 0.770 on 100 test rows
```

WHY A RANDOM FOREST

It is an **ensemble** of many decision trees whose votes are averaged — robust, low-maintenance, handles mixed encoded features, and resists overfitting better than a single tree. A strong default for tabular data.

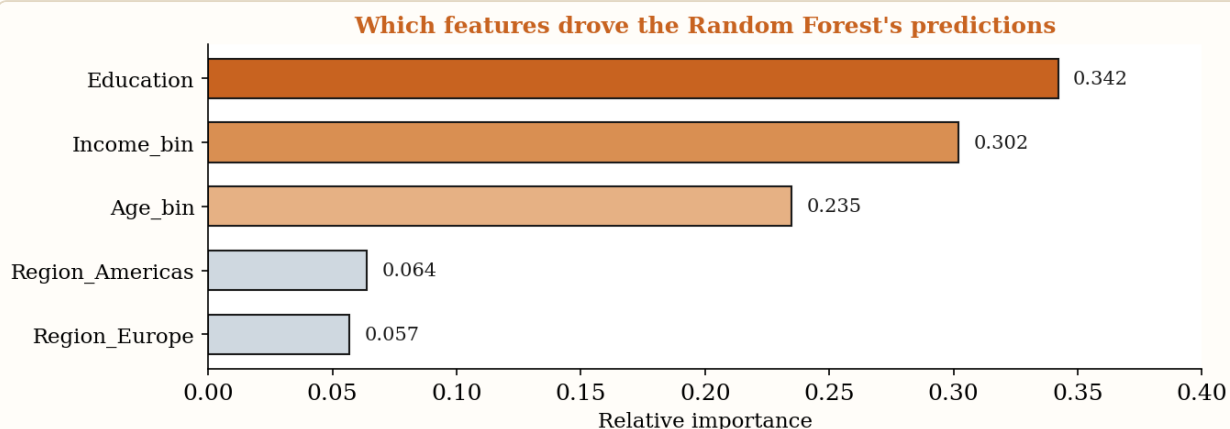
Verified output from the companion script (fixed seeds, so it reproduces exactly):

```
Target balance: {'no': 212, 'yes': 188}    Train rows: 300    Test rows: 100

Region_Americas  Region_Europe  Education  Income_bin  Age_bin
1.0              0.0            2.0        2.0         4.0
0.0              0.0            3.0        3.0         2.0
0.0              1.0            0.0        4.0         0.0

Test accuracy: 0.770 (on 100 test rows)
```

The forest can also report which features it leaned on most. Education, the binned Income, and binned Age carried the signal — matching how the synthetic target was built:



READ IT CAREFULLY

Feature importance shows what this fitted forest *used*, not what **causes** the outcome. High importance for Education does not prove education causes purchases.

8 Five things beginners get wrong

- 1. Fit on train only.** Fit encoders and the discretizer on `X_train`, then `transform` the test set. Fitting on everything leaks test information into training and inflates your score.
- 2. The dropped one-hot category isn't missing.** With `drop='first'`, that category is the *reference* — represented when all the other one-hot columns are 0.
- 3. Fake order is a real bug.** Label/ordinal encoding nominal inputs invents rankings that distance-based models will trust.
- 4. Binning is a trade, not a free win.** It can reduce overfitting or discard useful detail. Measure both ways.
- 5. Importance is correlation, not causation.** Useful for insight, not proof.

9 Quick reference

Goal	Tool	One-liner
Nominal → columns	OneHotEncoder	<code>OneHotEncoder(drop='first')</code>
Ordinal → ranked ints	OrdinalEncoder	<code>OrdinalEncoder(categories=[order])</code>
Target → ints	LabelEncoder	<code>le.fit_transform(y)</code> (targets; not a default for inputs)
Continuous → bins	KBinsDiscretizer	<code>KBinsDiscretizer(n_bins=5, encode='ordinal')</code>
Fit vs apply	fit / transform	fit on train , transform test
Classify	RandomForestClassifier	<code>rf.fit(X_train, y_train)</code>

Glossary — 104 terms

Every concept and term from this guide's companion notes, in plain language.

Accuracy — The proportion of test examples whose predicted class matches the actual class. It can be misleading when classes are imbalanced, so pair it with precision, recall, or F1 in those cases.

Age — A continuous numerical input feature in the demonstration. It is converted into five ordinal bins before model training.

Algorithm — A defined computational procedure used to learn patterns, transform data, or make predictions.

Area Under the ROC Curve (AUC) — A threshold-independent metric that measures how well a classifier separates positive and negative classes. Higher values indicate better discrimination.

Artificial Ordinal Relationship — A false ranking implied when unordered categories are represented by integers, such as assigning APAC=0, Americas=1, and Europe=2.

Bias Prevention — The use of suitable preprocessing so that category representation does not improperly favor, rank, or distort particular values.

Binary Data — Nominal categorical data containing exactly two possible values, such as yes/no or purchased/not purchased.

Binary Encoding — An encoding technique that converts each category into a binary code. It can reduce dimensionality compared with one-hot encoding, especially for high-cardinality variables.

Bin — A numbered interval or bucket into which a continuous numerical value is placed during discretization.

Cardinality — The number of unique categories in a categorical feature. Region in the demo has cardinality three.

Categorical Data — Qualitative data whose values identify groups, classes, labels, or attributes rather than measured quantities.

Category — One possible value of a categorical variable, such as APAC, Americas, or Europe.

Class — A possible target outcome in classification. The demonstration has two classes: no and yes.

Classification — A supervised-learning task in which a model predicts a categorical target.

Continuous Data — Numerical data that can take many possible values within a range, such as income or age.

Count Encoding — Another name for frequency encoding, where each category is replaced by its occurrence count or frequency.

Data Processing — The preparation, cleaning, transformation, and organization of raw data so it can be analyzed or used by a model.

Data Reduction — Methods that reduce the number of variables or dimensions while attempting to preserve useful information.

Data Type — A classification describing the nature of a value, such as numerical, categorical, continuous, discrete, nominal, or ordinal.

Decision Tree — A model that makes predictions by repeatedly splitting data according to feature values.

Dense Array — An array that explicitly stores every value, including zeros. The demo requests dense output from OneHotEncoder for easier viewing and DataFrame construction.

Dimensionality — The number of input features or columns in a dataset. One-hot encoding can substantially increase it.

Discrete Data — Countable numerical values that are distinct and separate, commonly represented by integers.

Discretization — The conversion of continuous numerical values into a limited number of discrete intervals or bins.

Distance-Based Algorithm — A method whose behavior depends on numerical distances, such as K-means or K-nearest neighbors. False numeric ordering can mislead these algorithms.

Dummy Encoding — A common name for one-hot encoding. In some usages, it specifically means using one fewer column than the number of categories.

Ensemble — A model composed of multiple individual models whose outputs are combined. A random forest is an ensemble of decision trees.

Equal-Count Bins — Bins designed to contain approximately the same number of observations. The demo uses the quantile strategy to form these bins.

Equal-Width Bins — Bins whose numeric ranges have the same width between the minimum and maximum values. This corresponds to the uniform strategy described in the course.

F1 Score — The harmonic mean of precision and recall. It balances false-positive and false-negative concerns.

Feature — An input variable used by a model to make predictions. Region, Education, Income, and Age are features in the demo.

Feature Engineering — The creation, selection, or transformation of input variables to improve modeling effectiveness.

Feature Importance — A random-forest estimate of how much each feature contributed to reducing prediction error across the trees. It is relative, not causal.

fit — A scikit-learn operation that learns parameters from training data, such as category mappings, bin edges, or model rules.

fit_transform — A scikit-learn operation that first learns transformation parameters from training data and then applies the transformation to that same data.

Frequency Encoding — An encoding technique that replaces each category with how often it appears. It can handle high cardinality but may cause collisions when categories have identical frequencies.

Global Target Mean — The overall average of the target variable. In regularized target encoding, it can be blended with category-specific means.

handle_unknown='ignore' — A OneHotEncoder setting that prevents an error when new data contains an unseen category. The unseen category is represented without activating a learned category column.

High Cardinality — A categorical feature with many distinct values. One-hot encoding such a feature can create too many columns.

Income — A continuous ratio-scale input feature in the demonstration. It is discretized into five quantile bins.

Inference — The use of a trained model or fitted transformer on new, unseen data.

Input Feature — An independent variable supplied to a model. It is commonly represented by X.

Interval Data — Numerical data with ordered values and meaningful differences but no true zero, such as Celsius temperature.

KBinsDiscretizer — A scikit-learn transformer that divides numerical features into bins and returns an encoded bin assignment.

Label — A target category that the model is trained to predict.

Label Encoding — The assignment of a unique integer to each category. It is commonly appropriate for a categorical target, but may create false ordering when applied to nominal input features.

LabelEncoder — The scikit-learn transformer used in the demo to map the target labels no and yes to 0 and 1.

Linear Discriminant Analysis (LDA) — A supervised dimensionality-reduction and classification method that uses class information.

Linear Regression — A supervised algorithm used to predict a continuous numerical target.

Machine Learning Model — A parameterized computational system that learns relationships from data and applies them to predictions or decisions.

Min-Max Scaling — A numerical preprocessing method that maps values into a fixed range, commonly 0 to 1.

Mixed Data — A dataset containing more than one kind of variable, such as text categories and continuous numbers.

Model-Ready Value — A numerical representation organized in a form accepted by the chosen machine-learning algorithm.

Multicollinearity — Strong linear dependence among input columns. Including every one-hot column plus an intercept can create exact redundancy.

Nominal Data — Categorical data with distinct categories but no meaningful order, such as geographic region.

Noise — Random variation or irrelevant fluctuation that obscures the underlying pattern in data.

Normalization — A family of transformations that adjust numerical scales. The term is often used broadly, but should be distinguished from standardization.

np.random.default_rng — NumPy's modern random-number generator constructor. The demo uses a fixed seed for reproducibility.

One-Hot Encoding — A transformation that creates binary indicator columns for categorical values so nominal categories are represented without ranking.

OneHotEncoder — The scikit-learn transformer used in the demo to convert Region into indicator columns.

Ordinal Data — Categorical data whose categories have a meaningful ranking, although the differences between ranks need not be equal.

Ordinal Encoding — A transformation that maps ordered categories to ordered integers while preserving their stated rank.

OrdinalEncoder — The scikit-learn transformer used in the demo to map High School, Bachelor, Master, and PhD to 0, 1, 2, and 3.

Ordinal Bias — Model distortion caused by incorrectly implying that nominal categories have a numerical order or distance.

Outlier — An observation far from most other values. Binning may reduce an outlier's influence, but it does not automatically make the observation unimportant.

Overfitting — Learning training-specific noise or accidental patterns that do not generalize well to unseen data.

Pandas DataFrame — A labeled two-dimensional table used to store and manipulate the dataset and transformed features.

Pattern Recognition — The identification of regular relationships in data that can support classification or prediction.

Precision — Among all cases predicted positive, the proportion that are actually positive. It reflects resistance to false positives.

predict — A model method that returns the predicted class for each input row.

predict_proba — A classifier method that returns estimated probabilities for each possible class.

Principal Component Analysis (PCA) — An unsupervised dimensionality-reduction technique that creates new continuous variables capturing major variance directions.

Pseudorandom — Generated by a deterministic algorithm that behaves like randomness. A fixed seed makes the same sequence reproducible.

Quantile — A boundary that divides ordered data according to proportions of observations.

Quantile Binning — Discretization that creates bins containing approximately equal numbers of observations rather than equal numeric widths.

quantile_method='averaged_inverted_cdf' — The explicitly selected NumPy-compatible quantile calculation method used by the demo's KBinsDiscretizer to determine quantile edges and avoid a future-default warning.

Random Forest — An ensemble classifier that trains many decision trees on varied samples and feature subsets, then combines their votes.

RandomForestClassifier — The scikit-learn implementation used in the demo. It trains 200 trees with a fixed random state.

random_state — A fixed seed or control value used to make randomized training or splitting reproducible.

Ratio Data — Numerical data with ordered values, meaningful intervals, and a true zero, such as income, height, or weight.

Recall — Among all actual positive cases, the proportion correctly predicted positive. It reflects resistance to false negatives.

Region — A nominal input feature in the demo with the categories APAC, Americas, and Europe.

Regularization — A method used to discourage a model or encoding from fitting noise too closely. In target encoding, smoothing is one form of regularization.

ROC Curve — A plot of true-positive rate against false-positive rate across classification thresholds.

Scaling — Adjusting numerical feature magnitudes so variables are more comparable for algorithms sensitive to scale.

Sparse Matrix — A memory-efficient structure that stores primarily nonzero values. It is useful when one-hot encoding creates many zeros.

Standardization — A numerical transformation that commonly centers values around zero and scales them to unit standard deviation.

Strategy='quantile' — The KBinsDiscretizer setting used in the demo to construct approximately equal-count bins.

Strategy='uniform' — The KBinsDiscretizer setting described in the course that constructs equal-width bins across the observed range.

Supervised Learning — Machine learning in which examples include known target values used to train the model.

Synthetic Dataset — Artificially generated data designed to reproduce controlled patterns for demonstration or testing.

Target — The outcome variable the model learns to predict. Purchased is the target in the demo.

Target Balance — The distribution of target classes. The demo contains 212 no cases and 188 yes cases.

Target Encoding — An encoding technique that replaces each category with a statistic derived from its target values, often the category's target mean. It can be powerful but is prone to leakage and overfitting without careful cross-validation and smoothing.

Target Leakage — The improper use of information from the target or future/test data during feature construction, causing unrealistically strong evaluation results.

Test Accuracy — Accuracy measured on held-out rows that were not used to fit the model.

Test Set — Data reserved for evaluating a fitted preprocessing pipeline and trained model on unseen examples.

Train-Test Split — The division of data into a training portion and a held-out testing portion.

Training Set — The data used to learn encoder parameters, bin boundaries, and model rules.

transform — A scikit-learn operation that applies parameters already learned during fitting. Test data should be transformed, not refitted.

Unseen Category — A category that appears in later data but was absent during fitting.

Unseen Data — Observations not used to train the model. Performance on unseen data is the main test of generalization.

X — The conventional symbol for the input-feature matrix.

y — The conventional symbol for the target vector.

Learn more

Official documentation for every tool used in the guide.

scikit-learn — Preprocessing user guide

scikit-learn.org/stable/modules/preprocessing.html

OneHotEncoder

[.../sklearn.preprocessing.OneHotEncoder.html](https://sklearn.preprocessing.OneHotEncoder.html)

OrdinalEncoder

[.../sklearn.preprocessing.OrdinalEncoder.html](https://sklearn.preprocessing.OrdinalEncoder.html)

LabelEncoder

[.../sklearn.preprocessing.LabelEncoder.html](https://sklearn.preprocessing.LabelEncoder.html)

KBinsDiscretizer

[.../sklearn.preprocessing.KBinsDiscretizer.html](https://sklearn.preprocessing.KBinsDiscretizer.html)

RandomForestClassifier

[.../sklearn.ensemble.RandomForestClassifier.html](#)

pandas — get_dummies (quick one-hot)

[pandas.pydata.org/docs/reference/api/pandas.get_dummies.html](#)

train_test_split

[.../sklearn.model_selection.train_test_split.html](#)

© 2026 David Grunwald. All rights reserved.